

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
```

```
# Load the MNIST dataset
(train_images, _), (test_images, _) = tf.keras.datasets.mnist.load_data()
```

```
# Preprocess the images
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5 # Normalize to [-1, 1]
```

```
# Create a TensorFlow dataset
BUFFER_SIZE = 60000
BATCH_SIZE = 256
train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 2s 0us/step

```
def make_generator_model():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Reshape((7, 7, 256)))
    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same', use_bias=False, activation='tanh'))
    return model
```

```
def make_discriminator_model():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same', input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Flatten())
    model.add(layers.Dense(1))
    return model
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
```

```
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss
```

```
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)
```

```
generator = make_generator_model()
discriminator = make_discriminator_model()
```

```
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

Warning: Do not pass an `input\_shape`/`input\_dim` arg
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Warning: Do not pass an `input\_shape`/`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
import tensorflow as tf
```

```
@tf.function
```

```
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])
```

```
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
```

```

fake_output = discriminator(generated_images, training=True)

gen_loss = generator_loss(fake_output)
disc_loss = discriminator_loss(real_output, fake_output)

gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

return gen_loss, disc_loss

def train(dataset, epochs):
    for epoch in range(epochs):
        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)

        print(f'Epoch {epoch + 1} completed - Generator Loss: {gen_loss.numpy():.4f}, Discriminator Loss: {disc_loss.numpy():.4f}')

    # Save the models after all epochs are completed
    generator.save('generator_final.keras')
    discriminator.save('discriminator_final.keras')

    # Print the last epoch and loss values
    print(f"Training completed after {epochs} epochs.")
    print(f"Final Generator Loss: {gen_loss.numpy():.4f}")
    print(f"Final Discriminator Loss: {disc_loss.numpy():.4f}")

# Assuming BATCH_SIZE and train_dataset are defined and generator_optimizer, discriminator_optimizer are initialized
EPOCHS = 100
train(train_dataset, EPOCHS)

```

```

Epoch 1 completed - Generator Loss: 0.7356, Discriminator Loss: 1.3063
Epoch 2 completed - Generator Loss: 0.9459, Discriminator Loss: 1.1303
Epoch 3 completed - Generator Loss: 0.7621, Discriminator Loss: 1.2961
Epoch 4 completed - Generator Loss: 0.8759, Discriminator Loss: 1.3244
Epoch 5 completed - Generator Loss: 1.1091, Discriminator Loss: 0.8300
Epoch 6 completed - Generator Loss: 0.7042, Discriminator Loss: 1.4789
Epoch 7 completed - Generator Loss: 0.9987, Discriminator Loss: 1.0596
Epoch 8 completed - Generator Loss: 0.7887, Discriminator Loss: 1.3723
Epoch 9 completed - Generator Loss: 0.7612, Discriminator Loss: 1.3782
Epoch 10 completed - Generator Loss: 1.1023, Discriminator Loss: 0.9731
Epoch 11 completed - Generator Loss: 1.1827, Discriminator Loss: 0.9120
Epoch 12 completed - Generator Loss: 0.8448, Discriminator Loss: 1.5028
Epoch 13 completed - Generator Loss: 0.9837, Discriminator Loss: 1.1223
Epoch 14 completed - Generator Loss: 1.0294, Discriminator Loss: 1.1758
Epoch 15 completed - Generator Loss: 1.0798, Discriminator Loss: 1.1922
Epoch 16 completed - Generator Loss: 1.1286, Discriminator Loss: 0.9259
Epoch 17 completed - Generator Loss: 1.0663, Discriminator Loss: 1.0969
Epoch 18 completed - Generator Loss: 0.8893, Discriminator Loss: 1.6163
Epoch 19 completed - Generator Loss: 1.3877, Discriminator Loss: 0.9712
Epoch 20 completed - Generator Loss: 1.2091, Discriminator Loss: 1.0317
Epoch 21 completed - Generator Loss: 1.1143, Discriminator Loss: 1.1228
Epoch 22 completed - Generator Loss: 1.3944, Discriminator Loss: 0.9325
Epoch 23 completed - Generator Loss: 1.2316, Discriminator Loss: 1.0719
Epoch 24 completed - Generator Loss: 1.2964, Discriminator Loss: 0.8151
Epoch 25 completed - Generator Loss: 1.1878, Discriminator Loss: 1.1483
Epoch 26 completed - Generator Loss: 1.0140, Discriminator Loss: 1.1896
Epoch 27 completed - Generator Loss: 1.2396, Discriminator Loss: 1.0323
Epoch 28 completed - Generator Loss: 1.1213, Discriminator Loss: 1.0027
Epoch 29 completed - Generator Loss: 1.2859, Discriminator Loss: 0.9837
Epoch 30 completed - Generator Loss: 1.2374, Discriminator Loss: 1.1067
Epoch 31 completed - Generator Loss: 1.0629, Discriminator Loss: 1.1287
Epoch 32 completed - Generator Loss: 1.0236, Discriminator Loss: 1.2085
Epoch 33 completed - Generator Loss: 1.0713, Discriminator Loss: 1.1546
Epoch 34 completed - Generator Loss: 1.0764, Discriminator Loss: 1.1161
Epoch 35 completed - Generator Loss: 1.0246, Discriminator Loss: 1.2672
Epoch 36 completed - Generator Loss: 1.1650, Discriminator Loss: 1.1510
Epoch 37 completed - Generator Loss: 0.9273, Discriminator Loss: 1.2032
Epoch 38 completed - Generator Loss: 0.9262, Discriminator Loss: 1.2778
Epoch 39 completed - Generator Loss: 0.9409, Discriminator Loss: 1.2666
Epoch 40 completed - Generator Loss: 1.1008, Discriminator Loss: 1.1732
Epoch 41 completed - Generator Loss: 1.0056, Discriminator Loss: 1.2744
Epoch 42 completed - Generator Loss: 0.9145, Discriminator Loss: 1.1271
Epoch 43 completed - Generator Loss: 1.0463, Discriminator Loss: 1.2404
Epoch 44 completed - Generator Loss: 1.2328, Discriminator Loss: 0.9736
Epoch 45 completed - Generator Loss: 1.4168, Discriminator Loss: 0.9104
Epoch 46 completed - Generator Loss: 1.0862, Discriminator Loss: 1.1295
Epoch 47 completed - Generator Loss: 0.9807, Discriminator Loss: 1.1444
Epoch 48 completed - Generator Loss: 1.0106, Discriminator Loss: 1.3470
Epoch 49 completed - Generator Loss: 0.9574, Discriminator Loss: 1.1403
Epoch 50 completed - Generator Loss: 1.0629, Discriminator Loss: 1.1177
Epoch 51 completed - Generator Loss: 0.8682, Discriminator Loss: 1.1696
Epoch 52 completed - Generator Loss: 1.1514, Discriminator Loss: 1.2083
Epoch 53 completed - Generator Loss: 0.9596, Discriminator Loss: 1.1380

```

Epoch 54 completed - Generator Loss: 0.8535, Discriminator Loss: 1.2452
 Epoch 55 completed - Generator Loss: 1.0038, Discriminator Loss: 1.2311
 Epoch 56 completed - Generator Loss: 1.0880, Discriminator Loss: 1.0446
 Epoch 57 completed - Generator Loss: 1.1552, Discriminator Loss: 1.0156
 Epoch 58 completed - Generator Loss: 1.1103, Discriminator Loss: 1.1656

```
import os
```

```
# List all files in the current directory
print("Current directory files:", os.listdir('.'))
```

```
Current directory files: ['.config', 'generator_final.keras', 'discriminator_final.keras', 'drive', 'sample_data']
```

```
# Load the saved models
generator = tf.keras.models.load_model('generator_final.keras')
discriminator = tf.keras.models.load_model('discriminator_final.keras')
```

```
# Verify the models are loaded
generator.summary()
discriminator.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

Total params: 2,330,944 (8.89 MB)
 Trainable params: 2,305,472 (8.79 MB)
 Non-trainable params: 25,472 (99.50 KB)
 Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

Total params: 236,656 (234.50 KB)

```
from google.colab import files
```

```
# Download the generator model
files.download('generator_final.keras')
```

```
# Download the discriminator model
files.download('discriminator_final.keras')
```



```
# Generate images
noise = tf.random.normal([16, 100])
```

```
generated_images = generator(noise, training=False)
```

```
# Evaluate discriminator
```

```
decision = discriminator(generated_images)
```

```
# Display generated images with discriminator's decision
```

```
plt.figure(figsize=(8, 8))
```

```
for i in range(generated_images.shape[0]):
```

```
    plt.subplot(4, 4, i + 1)
```

```
    plt.imshow(generated_images[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
```

```
    plt.title(f"'Real' if decision[i, 0] > 0 else 'Fake'\n{decision[i, 0]:.2f}")
```

```
    plt.axis('off')
```

```
plt.show()
```

